



Satakunnan ammattikorkeakoulu
Satakunta University of Applied Sciences

TEEMU HARJULA

Design and Simulation of an Open-Source Articulated Robot Using ROS 2

DEGREE PROGRAMME IN MECHATRONICS
2025

ABSTRACT

Harjula, Teemu: Design and Simulation of an Open-Source Articulated Robot Using ROS 2

Bachelor's thesis

Degree programme in Mechatronics

May 2025

Number of pages: 44

The objective of this thesis was to design and simulate a six-axis articulated robot, that could be used in robotics education and research. The focus was on developing an open-source robot model, that could be imported into Robot Operating System 2 environment to be simulated.

The frame structure of the robot was designed in SolidWorks and consists of simple to manufacture aluminium plate parts. Simple frame structure was used for supply chain resilience reasons, local manufacturing capabilities in mind. The robot model was then imported into ROS2 environment, where it was simulated in Gazebo and further, integrated with MoveIt2 motion planning package.

The result of the thesis was a working simulation setup in ROS2 Jazzy environment in Ubuntu 24.04. The system demonstrated motion planning in with MoveIt2 in RViz2 visualisation and the execution of planned motions in Gazebo simulation. Essential ROS2 packages were created and configured, including robot description and mesh files, simulated controller configurations and launch files. The final system can be used to demonstrate ROS2 implementation with custom robots.

It was concluded that the developed 3d model and simulation environment configurations provide a platform for teaching and demonstrating the process of using ROS2 in custom robot development. The thesis created a base for future work, including a potential physical manufacturing and further development of the simulation for educational use.

Keywords: Robotics, Robot operating system 2, Open source code

CONTENTS

1 INTRODUCTION	6
1.1 Background and motivation	6
1.2 Purpose and objectives	6
1.3 Scope and limitations	7
2 CURRENT STATE OF ROBOTICS	7
2.1 Fundamentals of modern robots.....	7
2.1.1 Joint types, kinematic chain and degrees of freedom	8
2.1.2 Robot workspace	8
2.1.3 Kinematics	9
2.1.4 Singularities	9
2.2 Types of robots in industrial use	10
2.3 Articulated robot tasks in industry.....	13
2.4 Articulated robot joint configurations	13
2.5 Collaborative Robots	14
2.5.1 Collaborative robot future	15
2.6 Use of simulation and visualisation in modern robotics	15
2.7 Future developments in robotics	15
3 DESIGN CONSIDERATIONS AND CHOICES	17
3.1 Design criteria	17
3.2 Frame type and material.....	17
3.2.1 Cast metal frame	17
3.2.2 Tubular frame	18
3.2.3 Experience with 3D printed frames	18
3.3 Motor Types and Drivers	19
3.3.1 Stepper motors	19
3.3.2 DC motors	20
3.3.3 Servo motors	20
3.3.4 Motor brakes.....	20
3.3.5 Motor selection	20
3.4 Control systems and communication.....	21
3.5 Gearboxes, drives and power transmission.....	22
3.5.1 Gears and gearboxes	22
3.5.2 Belts and pulleys.....	22
3.6 Bearing considerations	23
4 EXISTING OPEN-SOURCE PROJECTS.....	23

5 METHODOLOGY AND DEVELOPMENT PROCESS	24
5.1 Iterative design and virtual prototyping	24
5.2 Preliminary part selection	25
5.3 Frame mechanical design principles	26
5.4 SolidWorks design process	27
5.5 Preparing the model for SW2URDF Exporter	27
5.6 Exporting to URDF	29
6 IMPORTING THE ROBOT INTO ROS2	30
6.1 MoveIt2 setup assistant	32
6.1.1 MoveIt2 demo in RViz2	33
6.2 Simulating the robot in Gazebo	33
6.2.1 Setting up Gazebo simulation	34
6.2.2 Troubleshooting the Gazebo environment	34
6.2.3 Gazebo and MoveIt2 interaction	35
7 RESULTS AND CONCLUSIONS	37
7.1 Evaluation of design choices	37
7.2 Objectives, results and outcomes	38
7.3 Future development and application	39
7.4 Personal reflections and learning outcomes	39
REFERENCES	40
APPENDIX 1: OVERVIEW OF THE ROBOT	43
APPENDIX 2: GITHUB REPOSITORY OF THE PROJECT	44

LIST OF SYMBOLS AND TERMS

CNC: Computer Numerical Control.

DOF: Degrees of Freedom.

Gazebo: A 3D-simulation software.

ROS: Robot Operating System, versions 1 and 2. Open-source robotics middleware suite.

RViz2: A visualisation tool in ROS2.

STL: A 3D mesh file format.

URDF: Unified Robot Description Format. An XML file format used to describe robots in ROS.

XACRO: An XML macro language used to create more readable and maintainable URDF files.

1 INTRODUCTION

1.1 Background and motivation

Robotics is playing an ever-growing role in the industry across all fields. With a steady increase, amount of worldwide operational robots has increased from 1.3 million units (2013) to over 4 million units (2023) in the past ten years. (International Federation of Robotics, 2024.) This continuous increase in the number of operational robots is creating a need for more robotic expertise in the industry. In addition, the fast development of artificial intelligence and its effect on robotics and industry in general is changing the landscape of smart, efficient and sustainable manufacturing. (International Federation of Robotics, 2025.)

The general increase in the demand for robotics experts, and the evolving nature of smart systems currently, is pushing change into higher education curriculums. This need for multidisciplinary expertise is already apparent when looking at for example our own Mechatronics curriculum, where multiple fields of engineering are combined into one degree. My motivation for this thesis is to ultimately provide new tools for education in the field of open-source robotics. Expertise in open-source systems, including robotics, can also be seen as an act for local security of supply, i.e. supply chain resilience by reducing reliance in foreign vendors and service providers.

1.2 Purpose and objectives

The purpose of this thesis is to first overview the current state of robotics, basics of robot design and open-source projects in robotics, focusing on articulated robots, and further, design and simulate a 3D model of a six-axis articulated robot. With the 3D model, technical drawings will be produced, to enable

future manufacturing of the robot. The robot will be imported into Robot Operating System 2 (ROS2) environment, to be visualised and simulated.

The objective of the thesis is to create a detailed 3D model and a working simulation in ROS2 and Gazebo, including motion planning with MoveIt2 package in RViz2 visualisation environment. These can later be used for teaching and demonstration purposes. Secondary objective is to provide technical drawings and a bill of materials for a potential future manufacturing of the designed robot.

1.3 Scope and limitations

Scope of this thesis are the creation of the 3D model of a six-axis articulated robot in SolidWorks and simulations in ROS2 environment. Physical prototypes will not be made during thesis work and would be included in the potential future build of the robot. Design decisions regarding the robot, including materials, hardware choices and control systems, will be made with a limitation of final budget of 5000€ in mind, should the robot be built in the future.

2 CURRENT STATE OF ROBOTICS

2.1 Fundamentals of modern robots

In today's world, robots are everywhere and come in a variety of shapes, sizes and appearances. From precise industrial robots to 3D printers and humanoid robots to autonomous mobile robots, the scope of robotics today is broad. In industrial use, common types include for example SCARA, gantry, delta, mobile and articulated arm robots.

2.1.1 Joint types, kinematic chain and degrees of freedom

Robot construction consists of nearly rigid links, which are connected to each other with joints, that allow relative movement between the links. This structure is referred as kinematic chain of a robot. Robot joints are generally divided into two types: rotary (revolute) joints and sliding (prismatic) joints. For most types of robots, degrees of freedom of a particular robot are defined by the number of joints of that robot. (Craig, 2005, p. 4.) In this thesis, focus will be on a serial kinematic chain of a six revolute joint articulated arm.

2.1.2 Robot workspace

Robot workspace is defined as the reachable positions for the end effector of the robot (Figure 1). This can be further be defined with the kinematic solutions for a point: if there are no kinematic solutions to reach a point, that point is not in the robot workspace. Robot geometry, especially link lengths determine the shape of the robot's workspace. (Siciliano & Khatib, 2016, p. 136.)

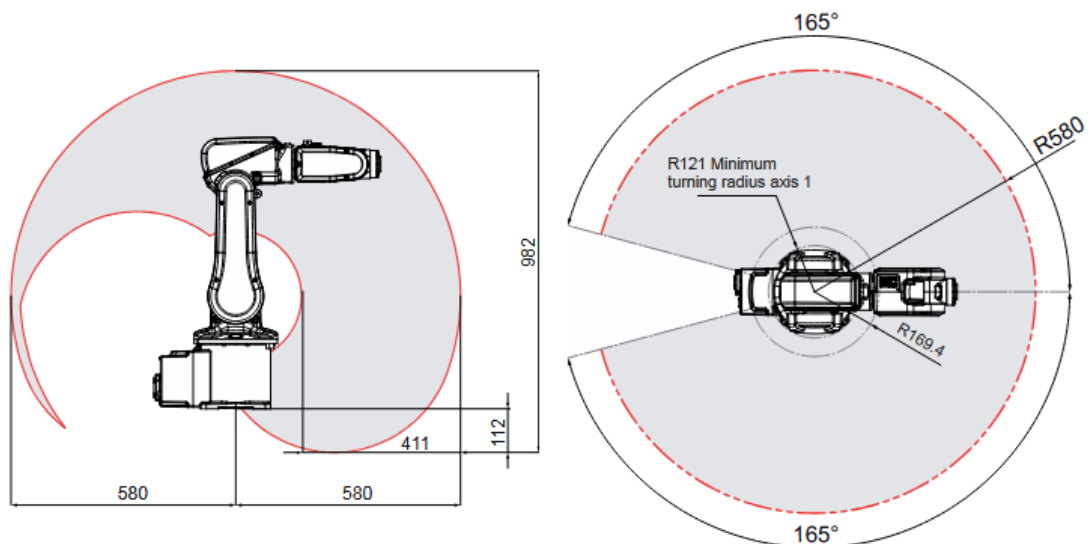


Figure 1. ABB IRB 120 Workspace viewed from the side and top. (ABB Robotics (2019).

2.1.3 Kinematics

For most end effector positions and poses, there are multiple joint positions that can be used to achieve the desired pose. Reaching those joint positions and effector poses requires kinematic calculations. Robot kinematics and kinematic calculations are divided into two categories: direct i.e. forward kinematics and inverse kinematics. With direct kinematics, end effector position in space is calculated from the actuator positions: joint angles (for revolute joints) or joint offsets (for prismatic joints). (Siciliano & Khatib, 2016, p. 137.) Compared to direct kinematic calculations, solving the problem of multiple linked actuator positions to achieve a desired end effector position is more challenging. For example, for a robot with six degrees of freedom, inverse kinematic calculations include 12 equations and six unknowns. (Craig, 2005, p. 101.) Kinematic calculations of the simulated robot in this thesis will be done by the MoveIt2 package in ROS2.

2.1.4 Singularities

Singularities refer to certain combinations of joint positions of the robot, where the end effector is forced to stop, while some of the robot joints are moving at high speed, to cross the singularity. In theory, crossing the singularity without stopping the end effector, would require infinite joint velocity. When two or more of its joints align, it causes the robot to lose one or more of degrees of freedom for the end effector. Robot singularities are typically divided into three categories by joints: wrist, elbow and shoulder singularities (Figure 2). Singularities can also be categorized into workspace interior and workspace boundary singularities. In practise, working in close proximity to a singularity, or passing through one, can result in end effector stops, increases in joint velocities and unexpected behaviours, which can result in an error. (Craig, 2005, pp. 9, 220; Owen-Hill, 2022a.)

Wrist singularity occurs when joints 4 and 6, either become coincident or parallel, depending on the robot wrist configuration. Wrist singularity results in the end effector stopping while joints 4 and 6 rotate into opposite directions at high

speed. Elbow singularity occurs when the center of the wrist is in the same plane with joints 2 and 3 (elbow and shoulder), i.e. the elbow is fully extended. Elbow singularities are an example of workspace boundary singularities. Lastly, shoulder singularity occurs when the center of the wrist aligns with the joint 1 (base), causing the end effector to stop while joints 1 and 4 rotate at high speed (Owen-Hill, 2022a.) For the scope of this thesis, acknowledgement of singularities, the different types and the causes are enough and a deeper look into the mathematics behind singularities is unnecessary.

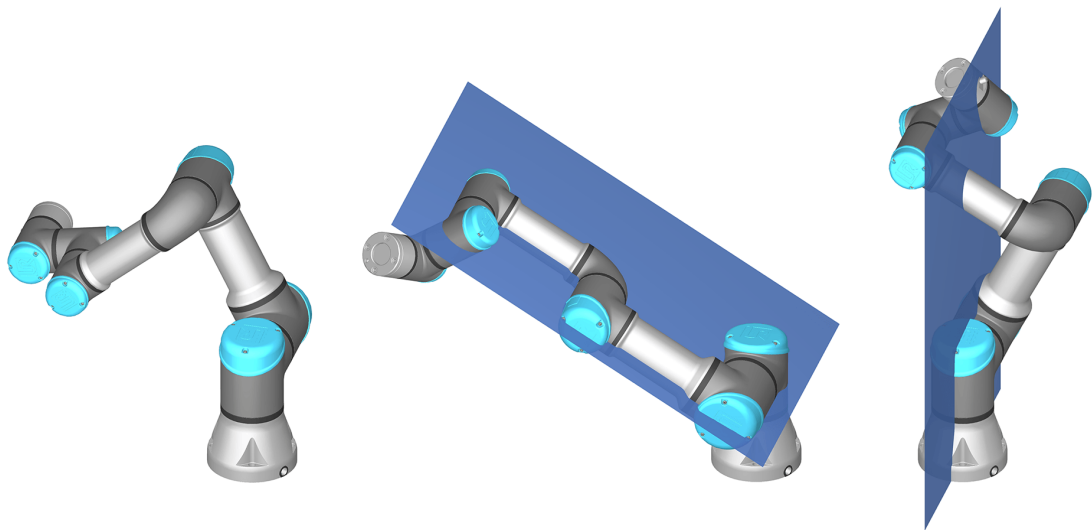


Figure 2. Different singularities examples with a non-spherical wrist robot. From left to right: wrist, elbow and shoulder. (Mecademic Industrial Robotics, n.d.).

2.2 Types of robots in industrial use

As mentioned in Section 2.1 regarding robot diversity, there are various types of robots, and this is also true for industrial use. The most recognizable one is the articulated arm robot, the most common being the six-axis variant. However, there are also articulated robots with fewer axes, as well as articulated robots with more than six axes. Another common robot type in the industry is SCARA (Selective Compliance Assembly Robot Arm) (Owen-Hill, 2022b.) A SCARA robot commonly consists of three rotational and one linear axis as seen in Figure 3.

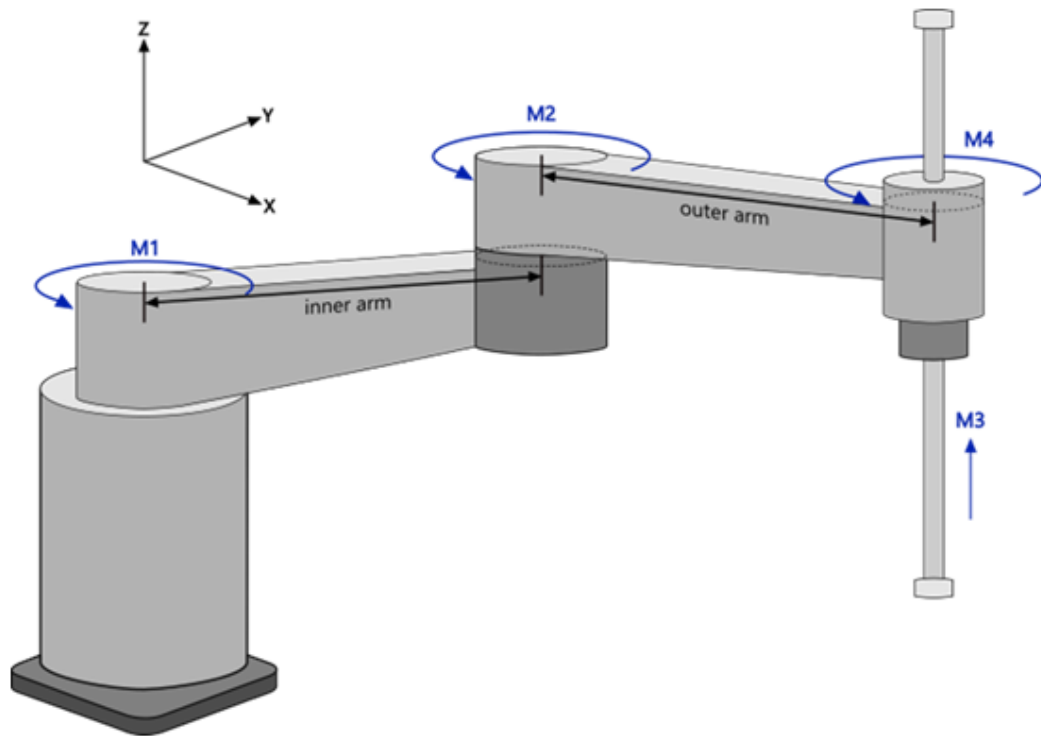


Figure 3. Typical SCARA robot configuration. Rotational axes are marked with M1, M2 and M4 and the linear axis with M3. (Kingstar, n.d.).

Other common robot types in industry include delta (parallel) robots and Cartesian/gantry robots. Delta robots are very fast and precise with a complex workspace due to their kinematic system, which consists of usually three parallel actuators (Figure 4). Delta robots are used for example in high-speed pick and place tasks. Gantry and Cartesian robots' kinematic system usually consists of three linear axes. While both gantry and Cartesian robots refer to the similar kind of movement system, the term gantry robot is generally used for physically larger robots, while Cartesian robot usually refers to a smaller one. Large gantry robots can be used in pick and place tasks or they can, for example, be fitted with an articulated robot as their 'end effector' to enable the use of the articulated robot in the whole workspace of the gantry robot. (Owen-Hill, 2022b.) Cartesian robots are used for example in PCB manufacturing, CNC-cutting machines, like plasma or laser cutters. Commercial 3D printers are a common example of a Cartesian robot. The movement system of the Cartesian robot makes it simple to perform tasks in a plane by only using two of the three available axes, while keeping the third fixed, for example in laser cutting. (Bernier, 2021.)



Figure 4. ABB IRB 390 FlexPacker delta robot. (ABB Robotics, 2020).

In industry today, there are also autonomous mobile robots (AMR) handling the transport of inventory in factories. These autonomous mobile robots are becoming increasingly common due to their high impact to the whole production. There are various types of AMRs, they can be more simple movement platforms that other operators place objects on, they can be used to pull trolleys or even be fitted with stacking capabilities, essentially becoming an autonomous forklift. Autonomous mobile robots often use SLAM (Simultaneous Localisation and Mapping) algorithms to safely navigate in a changing environment. (ABB Robotics, 2024.)

There is also a new type of robots entering the industrial environment, humanoid robots. While humanoid robots have traditionally been mostly used as social and service robots, technology advancements have made humanoid robot use in industry a probable near future. Humanoid robots main selling point is, as the name suggests, the human-like dexterity and the ability to be used in a very wide range of tasks. (Hanbury et al., 2025.) In my opinion, humanoid robots are still quite far away from competing with current specialised robots in single tasks, but I also understand the value in a true multipurpose robot that can work in different environments, especially as the prices of humanoid robots decrease.

2.3 Articulated robot tasks in industry

In modern industry, articulated robots are used in a wide variety of tasks. Common uses are material handling, palletizing, machine tending, welding and many more. Modern articulated robots are versatile machines: just by changing the end effector of the robot, they can be used a very wide range of tasks in the industry. Key performance metrics of modern articulated robots are high degree of accuracy and repeatability regarding position and movement path, sufficiently fast operating speed, payload capacity, dexterity and ease of use by modern and intuitive teach pendants and software solutions.

2.4 Articulated robot joint configurations

Modern articulated robots' joints can be arranged in different ways, which in turn can affect the structural properties and the complexity of kinematic calculations needed for motion planning. Two common categories are: the traditional industrial robot style, where all the joints are situated in-line with each other. The nature of this design results in simpler inverse kinematic calculations and allows for more rigidity, enabling higher speeds and accelerations. The second category is the more modern 'cobot' arrangement, where the joints are offset, resulting in a more versatile wrist in certain positions (Figure 5). A non-spherical wrist also results in more complex inverse kinematic calculations. (Boschetti & Sinico, 2023, pp. 15–16.)

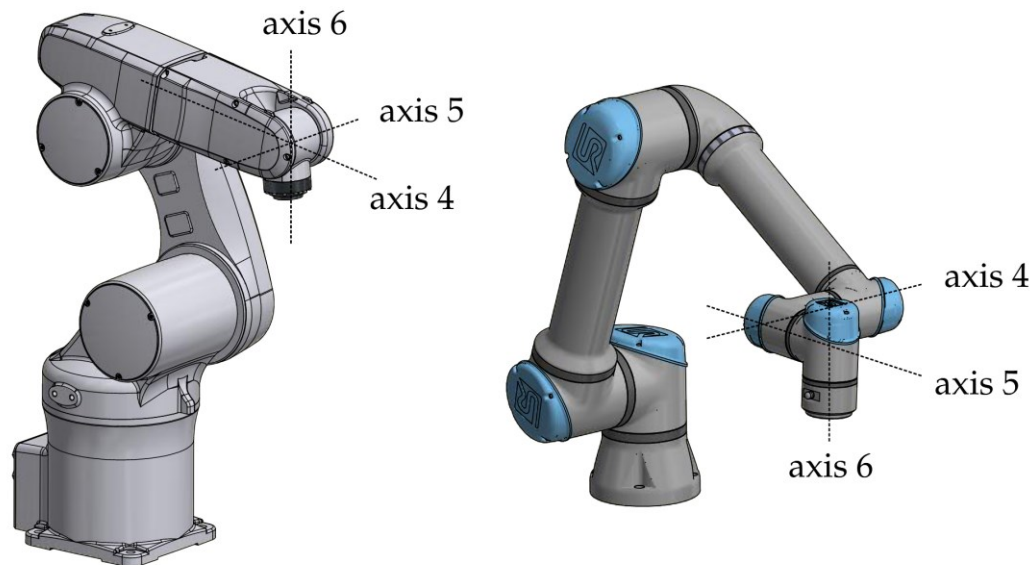


Figure 5. Comparison of spherical (left) and a non-spherical (right) wrist configuration. (Mecademic Industrial Robotics, n.d.).

2.5 Collaborative robots

When an articulated robot is designed to be used as a collaborative robot (cobot), there are trade-offs when compared to traditional industrial robots. Cobots include more safety sensor technology, for example force sensors in joints, which enable the use of robots in a shared workspace with human operators. This collaborative workspace and the advanced safety features of modern cobots allows better utilization of workspace. Where traditional industrial robots need a closed security perimeter for the defined workspace, cobots can safely operate without a fixed perimeter or enclosure, even in the same space with human operators. The greatest trade-off for the versatility of a modern cobot is the operating speed. Cobots are generally rated for slower speeds and payloads compared to similar size industrial robots, even when used independently without the need for collaborative features. For collaborative use, the robot is in Power and Force Limiting mode (PFL), which further limits the velocity of the robot to ensure limited forces in case of a collision. (ISO 12100:2010(en), p. 72; Malm, 2022, pp. 8, 18, 27.) While there are some proximity sensor solutions emerging, current cobots today mostly rely on force feedback on the robot joints. Because of the nature of these safety systems, the collision is only detected after the fact, so the inertias of the robot and

workpiece being handled must be kept reasonable in collaborative use. (Wang, 2022.)

2.5.1 Collaborative robot future

It is also important to note, that in the EU region, regulations on collaborative robots are changing. Until now, the 'collaborative' nature of a robot has been assessed by the robot manufacturer and the risk assessment of the cobot has been based only on the robot itself. From 2027 onwards, EU's new Machinery Regulation will change the risk assessment process of a collaborative system to assess the system as a whole and not just the collaborative robot. Also, the regulation change is bringing forward new design considerations. The regulation emphasizes for example cyber security of the control systems and safety implications of artificial intelligence used in the robot's operation. (Nimtz & Fichtner, 2024; Regulation 2023/1230/EU.)

2.6 Use of simulation and visualisation in modern robotics

Simulation is an important tool and is used in multiple areas of robotics nowadays. From robot design and development to programming and simulating robotic cells and further, using a digital twin of a real-world system to predict and evaluate for example maintenance need. Simulation and visualisation are used in many stages of robot system, from development and deployment to monitoring during runtime. (Choi et al., 2021.)

2.7 Future developments in robotics

Predicting future trends and innovations is of course very difficult. During my relatively brief time in robotics, I have seen the latest trends that are currently on the rise, such as integration of artificial intelligence into the systems, humanoid robots and autonomous mobile robots in everyday life, like grocery delivery robots. In my opinion, especially in the EU area, the near future will drive the robot development further into a more sustainable direction, both

from manufacturing and end use perspectives. While there are very competitively priced options available from Asia, I think the sustainability and supply chain resilience reasons will drive more robot development and manufacturing into Europe, which would decrease the dependence on Asian and American system providers. Especially for newer companies without pre-existing proprietary software, I think that open-source resources and frameworks, like ROS2, are an invaluable tool in the development process. Using existing libraries and frameworks can save time and resources on design and implementation of the general environment, hardware drivers and compatibility, thus allowing those resources and time to be allocated into real innovation, prototyping and development of new solutions.

Personally, I think that humanoid robots are currently overhyped, for what they provide in their current state. However, I also see the possibility of that changing radically, even very quickly, through innovation. When the humanoid robots reach truly multipurpose capabilities, without extensive setup per task, through AI integration and especially when their price decreases, I can see their usage increasing rapidly. In my eyes there are two main reasons for this: humanoid robots seem to attract most attention from the general public and second, most of the systems in our current environment are designed to be used by humans, and humanoid robots naturally fit that environment very well. I would argue that there are superior designs for most individual tasks when compared to a humanoid robot, but those designs often lack the versatility that a future humanoid robot could offer.

Also, as the sensor technology and artificial intelligence development moves forward and more data is collected from robot systems, I see the use of predictive maintenance being adopted into more and more industrial systems, not strictly limited to robotics, but also on broader system or even factory level. In summary, while I would like to see more fundamental mechanical innovation in robotics, I think the near future will make the whole industry, including robots themselves, even smarter, more interconnected and, more efficient and sustainable. As I mentioned in the beginning of this chapter, predicting the future

is difficult, and that is especially true in a field like robotics, where innovation can bring changes quickly and in many perspectives.

3 DESIGN CONSIDERATIONS AND CHOICES

3.1 Design criteria

Considering the budget limitation of the final assembled robot, there are multiple choices that must be made regarding the design of the robot. Different types of motors, gearboxes, bearings and control systems vary widely in price, performance and quality, and all parts need careful consideration to achieve a good price to performance ratio. From performance metrics regarding this particular robot design, safety, rigidity, accuracy and repeatability will be considered most important.

3.2 Frame type and material

The robot frame can be designed in various ways. There are advantages and disadvantages with different design and material choices. These choices affect not only the features of the final robot, but also the manufacturing process. By making smart design and material choices that consider the available tools and processing techniques, the process of manufacturing can be made relatively straightforward.

3.2.1 Cast metal frame

Many large, name brand industrial robots are made with custom cast metal parts. In high volume manufacturing, permanent mold casting is an efficient process when compared to stock removal machining processes. In small scale manufacturing though, permanent mold casting is expensive due to the high price of the molds. (Sunrise Metal, 2024.) While expendable mold casting

methods could be considered from cost standpoint, for the possible future manufacturing, there will be no facilities available to cast metal parts.

3.2.2 Tubular frame

Another common frame style in modern robots is the tubular frame that can be seen especially in many collaborative robots, such as Universal Robots lineup. In this frame style, the links between joints are essentially tube stock with interfaces for joint modules to attach to. This series of tubular links and joint modules form the articulated arm, where the shoulder-elbow and elbow-wrist links are often offset. Tubular frames also provide an easy way to design modularity into the robot. For example, robot geometry can be altered simply by changing the length of the links between the joints. (Cohen et al., 1992, p. 118.)

3.2.3 Experience with 3D printed frames

There are also some robot designs that are constructed from 3D printed parts, especially in the hobbyist and open-source space. Having experience with one of such robots (BCN3D Moveo), I would consider using 3D printed plastic parts in the main frame of the final robot frame sub-optimal, although they can offer large value when prototyping via time and money saving. In my experience, 3D printed plastic parts suffer from lower mechanical properties when compared to machined aluminium, by having lower tolerance for strain before elastic and plastic deformation.

For this design, a machined metal plate frame will be used. Material for the links will be aluminium due to being a good middle ground in mechanical properties, machineability and price. Other options would be steel, titanium and carbon fiber; steel being heavy, titanium being both hard to machine and expensive and carbon fiber, although very light and rigid, being also expensive and harder to machine when compared to aluminium.

3.3 Motor types

There are multiple options when selecting a motor for a custom robot: stepper motors, servo motors, DC motors and all-in-one robot actuators that incorporate the motor, motor driver and gearbox into one unit. Different motor and actuator options have their benefits and disadvantages, regarding accuracy, torque, price and so on. Different motor types also require different motor drivers and control logic to work. (Niryo, 2024.)

3.3.1 Stepper motors

Stepper motors are a commonly used option for low budget designs, but their inherent problem in robot design is the nature of the motor's movement. As the name suggests, stepper motors move in steps (Figure 6). And although they have improved over the years, still for example modern, budget friendly, NEMA 23 stepper motors from omc-stepperonline.com have a step angle of 1.8 degrees. This step angle can be reduced with micro stepping, which leads to lower holding torques in the motor. (Walter, 2016.)

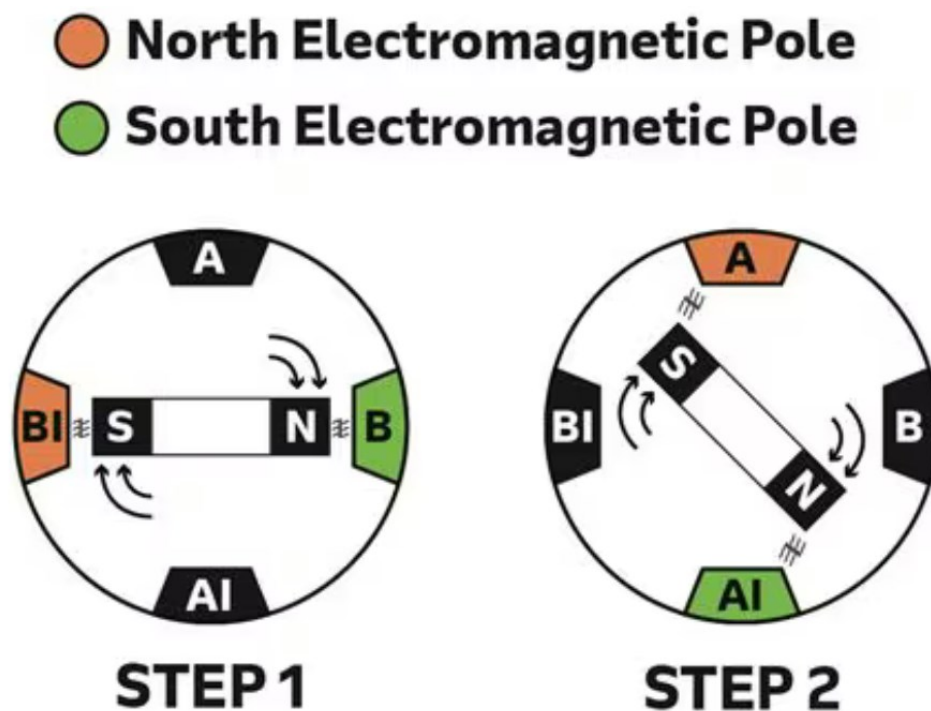


Figure 6. Stepper motor working principle with a simplified four step motor. Polarities of the outer coils are changed to attract the rotors magnetic poles. (Arrow Electronics, 2022).

3.3.2 DC motors

DC motors provide accurate speed with simple control at an affordable price. On brushed DC motors, there is always mechanical wear on the brushes themselves, that affect the lifespan and service intervals of the motor. Brushless DC (BLDC) motors provide longer lifespan, low maintenance and high efficiency. (Sntoom, 2024.)

3.3.3 Servo motors

Servo motors can be viewed as actuators, instead of just motors, due to the built-in positional sensor. Servo motors with their dedicated controllers offer a closed-loop control system, which makes servo motors good for maintaining accurate positioning even under varying loads. Both DC and AC powered servo motors can be found. (Sntoom, 2024.)

3.3.4 Motor brakes

In an articulated robot, it is important to have brakes in the actuators of the joints. Brakes enable powering off the robot when the arm is suspended, removing the need for placing the robot on a support before powering off. Second, the brakes, especially in the lower joints, are an important safety feature. For example, servo motor brakes are unlocked by powering them on, so in case of a power outage, the brakes will be activated automatically, prevent the robot from collapsing. (Rocco, 2018.)

3.3.5 Motor selection

There are also all-in-one robot actuators that combine the motor, reducer and sometimes even the controller in one package. At least from mechanical design standpoint, these all-in-one actuators would be the easiest solution. Due to their higher price and availability, DC servo motors with brakes will be used in this design.

3.4 Control systems and communication

As briefly mentioned in the previous chapter, motors can be either open or closed-loop controlled. In closed-loop control system, some sensor measuring the actual state of the motor, is used as feedback in the motor controller (Figure 7). Good example of a closed-loop motor control system is keeping a constant motor speed in varying loads by adjusting the motor input parameters with the data from the output speed of the motor. (Devasia, 2023.)

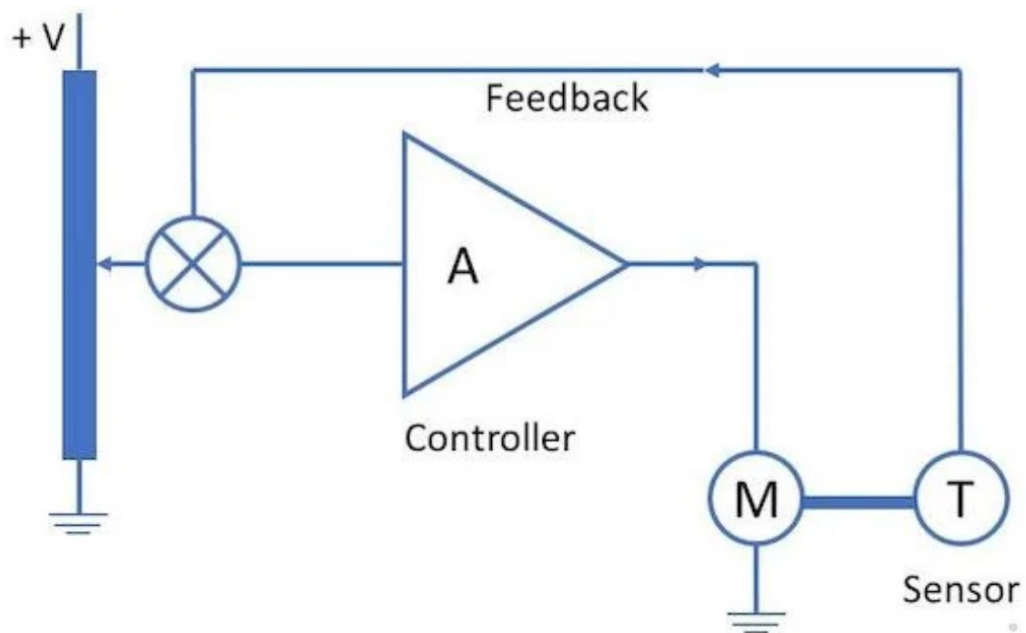


Figure 7. An example of a simple closed-loop motor control system, showing sensor T readings used as feedback in the controller A. (Devasia, 2023).

In my opinion, for robotic applications, open-loop control without positional feedback is not an option, when accuracy and positional repeatability are considered important. For this design, closed-loop control will be used, utilizing the DC servo motors built in encoders. These incremental encoders are magnetic with 17-bit resolution, resulting in 131072 measurements per revolution. The integrated incremental encoders do not work for initial position after system power-up, as they do not provide absolute positional data. For initial position of the joints on start-up, affordable absolute encoders will be used on each joint.

3.5 Gearboxes, drives and power transmission

Electric motors tend to operate more efficiently at higher speeds and delivering lower torques, than is needed in robot applications. Using speed reducing transmissions in robot systems to achieve the required speed and torque is common. (Hale, 1999, p. 243.) Speed reduction and force transmission in this robot design could be achieved by several ways: belts and pulleys with reduction ratio, gear-based transmissions and so-called no-backlash gearboxes including strain-wave gear boxes and cycloidal gearboxes.

3.5.1 Gears and gearboxes

Gears and different gearboxes are a common way to do speed reduction in mechanical systems. There are multiple ways to use gears for reduction, spur gears with parallel shafts, bevel gears, and worm gears to name few. Gears can also be in gearbox configurations, like planetary gearboxes (Figure 8). Gear-based systems have an inherent problem with backlash, reducing final accuracy of the movement system. Backlash arises from imperfect meshing of the gears and can be measured from the angular movement on the output side, while the input stays in place. (Craig, 2005, p. 245.)

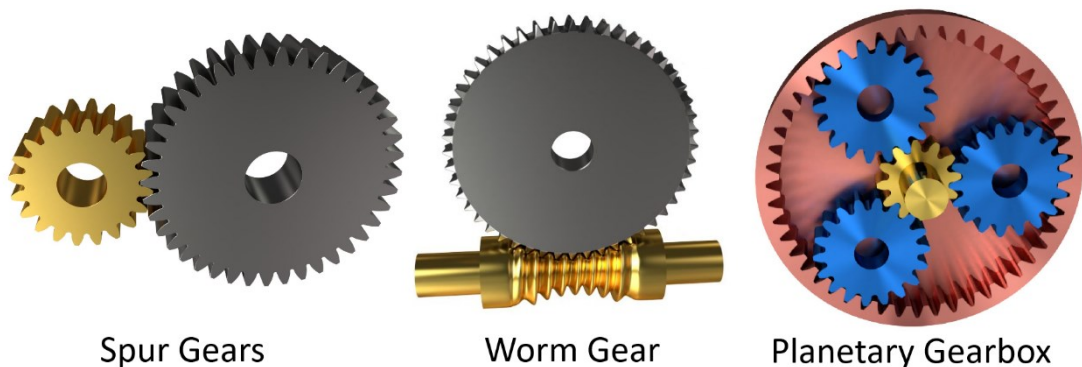


Figure 8. Some commonly used gear reduction options. (Tec-science, 2018).

3.5.2 Belts and pulleys

Belts and pulleys are another commonly used option, that can offer both speed reduction and power transmission further away from the motor. Belts in these

systems must be flexible, to travel across the rotating pulleys. This flexibility in the belts is also present in the longitudinal direction, although much less, and must be accounted for with proper tensioning. Modern belts use fabric or metal fibers in their structure to combat the longitudinal flexibility. (Craig, 2005, p. 246.)

The final design of the robot uses belts for power transmission in three joints. The main reasons for this are to keep the weight of the motor lower in the robot arm in joint 3, and to allow more convenient motor placement in joints 1 and 5. Two of the belt actuated joints also have harmonic drive in the joint, so no pulley-based reduction is necessary. In joint 5, a 5:1 reduction ratio will be implemented with different size pulleys.

3.6 Bearing considerations

Joints in articulated robots are subject to forces from different directions. Different types of bearings are used to ensure low friction and achieve smooth operation in different orientations and under varying loads. The forces on the robot joints can be divided into radial and axial forces. Radial loads can be mitigated with basic ball or roller bearings and axial loads with thrust bearings. There are also tapered bearings that counteract forces in both directions, but like thrust bearings, need to be preloaded. (Allinson, 2024.) The strain wave reducers in this design include a crossed roller bearing supporting the output flange, acting as an output bearing, reducing external bearing requirement at the reducer output side.

4 EXISTING OPEN-SOURCE PROJECTS

There are various existing open-source projects already in the robotics field, ranging from very affordable tabletop robots to higher performance robots used in education, research and automation. As mentioned earlier, during one

of the courses we assembled a BCN3D Moveo robot, which is one of these open-source projects. For me, it highlighted some of the design flaws as well as some shortcomings due to component and material selections. During that project, it became clear that achieving high performance with a robot, needs careful consideration throughout the design process and for example open-loop control systems are not a good option. Also, the use of 3D printed parts for the main structure, while a very convenient, easy and affordable option, is not ideal. After going through several different open-source projects, there were no good options that combined a metal frame, six degrees of freedom (6 DOF) and closed-loop servo motors with low backlash gearboxes (Table 1). This is probably due to price: both, the price difference between stepper and servo motors and the high price of harmonic and cycloidal reducers in general.

Table 1. Some popular open-source projects compared

Robot	DOF	Material	Motors	Reduction	Control
BCN3D Moveo	5	3D printed plastic	Stepper	Belt reduction	Open-loop
Annin AR4	6	CNC-Aluminium	Stepper	Planetary	Closed-loop
Arctos	6	3D printed plastic	Stepper	3D printed cycloidal, belt reduction	Open/close options
Parol6	6	3D printed plastic	Stepper	Planetary, belt reduction	Closed-loop

5 METHODOLOGY AND DEVELOPMENT PROCESS

5.1 Iterative design and virtual prototyping

The robot design process started with a general high-level idea which was then very early brought into a 3D modelling software, in this case SolidWorks 2025. This is simply because it is a very natural way for me personally to drive the design forward and see possible problems as they arise, when the modifications are visible in real time and in 3D. This preliminary design process iterates

quickly; the changes can be aggressive and there may be multiple versions simultaneously to compare and to extract the best features from. This method of designing is very closely connected to the CDIO (conceive, design, implement, operate) principles, that have been introduced to my workflow during schoolwork.

5.2 Preliminary part selection

Early in the design process, preliminary choices were made for potential motors, drives, transmission components and bearings, around which to then start the actual designing. An important aspect of the component choices was to utilize parts and materials that are commonly available, to make sourcing parts for future build easier. The starting point was 200W and 400W DC servo motors and two different sizes of strain wave gearboxes, and most of the original parts stayed in the design. The final component list regarding motors and gearboxes can be seen in Table 2.

Table 2. Main components of the robot.

Part	Quantity	Position	Info
400W Servo with break	3	J1, J2, J3	24-70V DC
200W Servo without break	3	J4, J5, J6	24-70V DC
110mm Harmonic drive	1	J1	50:1, 1.22kg
90mm Harmonic drive	2	J2, J3	50:1, 0.72kg

The servo motors selected for the robot come with dedicated servo drivers and the motors can be controlled by RS485 bus. The motors are fitted with 17-bit incremental encoders for accurate positional closed-loop control. Two different sizes of 50:1 harmonic drive were used as reducers. As the joint 1 is exposed to most mechanical stress, a larger diameter 110mm drive was selected, simply for higher force tolerance in the drive assembly, due to the larger crossed roller bearings on the output side of the drive. For the joints 2 and 3, smaller diameter 90mm drives were selected, they were a good compromise between lower mass higher in the robot arm, and still adequate force tolerances. All the selected harmonic drives have maximum average input speed

of 3500rpm, which combines well with the rated speed of 3000rpm of the servo motors. Joints 3, 4 and 5 are driven with servo motors without gearboxes due to the lower torque requirements. Belt drives are used in joints 1, 3 and 5 to both keep the mass of the motors as low in the arm as possible and allowing for more convenient motor placement on the frame.

5.3 Frame mechanical design principles

After the preliminary selection for the main parts, the actual design process started with the idea of building a frame with tubular links and machined joint modules to house motors and gearboxes. After a careful consideration for the future manufacturing, and especially, the machining and manufacturing capabilities at our disposal in the Satakunta University of Applied Sciences mechanical laboratory, it became clear that the tubular frame design required more advanced 4/5-axis CNC-milling, which we do not have access to in-house. While the advantages of the tubular frame are clear when considering structural strength of the circular profile and the possibility for modularity, manufacturing these parts accurately, without appropriate machinery would be very challenging.

The frame design was restarted with different approach, with ease of manufacturing in mind. The final design is constructed out of 12mm aluminium plate that can be cut to shape with a water, laser, or plasma cutter, with features that can be machined on a normal 3-axis CNC-mill. The parts can also be milled on a manual 3-axis mill, but there are features that will require some more exotic work holding or tooling. While the rounded outer features of the links are not dimensionally critical for the assembly, optional finishing passes on them may require a rotary indexing table or a vice. The inner bores in the parts, that support the motors, bearings or gearboxes must be machined accurately. The same rotary indexing fixture could be used to machine the larger bores, or an adjustable diameter boring bar could be used.

5.4 SolidWorks design process

The frame was modelled one joint-to-joint link at a time, in a SolidWorks assembly file, creating new SolidWorks parts for each link. While working with the whole assembly can become more demanding for the hardware as the assembly gets more complex, it provides couple of clear benefits from parametric design perspective. First, it makes using global equations and variables simple for all the parts in the assembly. Changing one variable on assembly level, affects and refreshes all the instances of that variable, on every part inside the assembly. There is also an exporting and importing feature for these equations and variables on a part level, but I found the procedure inconvenient, while the assembly level equations and variables are simple to use and work well. The second benefit of modelling in an assembly, is having a so called 'skeleton sketch' that contains the most important dimensions of the robot, and some geometry that can later be used for mating parts within the assembly. The 'skeleton sketch' can be dimensioned with the global variables, simplifying the modification of the robot later in the design.

The mates between parts in the assembly can be setup in a way, that the robot arm is functional, and can be done even with movement i.e. angle limits, that restrict the movement range or the robot's joints. Having functional mates in the assembly makes it easy to determine self-collisions within the frame, when refining the frame geometry. This ability to move the robot around in 3D space makes it straight forward to notice possible problem areas in the design, enabling the fast iteration design process that was mentioned in the opening of chapter 5.

5.5 Preparing the model for SW2URDF Exporter

SolidWorks to URDF Exporter (SW2URDF) is a free plugin for SolidWorks, that is authored and maintained by Stephen Brawner and can be found on GitHub. The plugin is an easy graphical way to generate URDF files to be used with ROS in visualisations and simulations. Officially, the plugin supports

SolidWorks versions from 2018 to 2021, but in my experience, it worked as intended also in SolidWorks 2025, which was used as the main modelling software.

Before using the plugin, some preparations must be made in the model. First, the robot should be fixed to a reasonable position for defining the joints and links for the plugin. A 90 degree 'home position', seen in Figure 9, was used in this case. Second, there must be a global coordinate system to reference. All links need to have a coordinate system which aligns with the global coordinate system's axes. Also, all joints need an axis; in this case all joints are 'revolute' type, so the axis will be the axis of rotation for the joint. Further, each joint axis must be coincident with a link coordinate system axis. Lastly, if a material has not been set for the parts in the model, it is good to do so here for inertia calculations happening during the exporting process, 6061 aluminium alloy was selected for the frame of this robot.

It is also good practice to use separate mesh files for the visual and physical parts of the simulation; these are referenced as visual and collision files in URDF format. This separation allows for high visual fidelity with visual mesh files, and good performance with much simpler collision mesh files for collision calculations in later movement planning operations (Figure 9). The collision parts were created simply by sketching and extruding a bounding box around each part, in the same assembly file. The collision files can also be created separately, but ultimately, they should be added to the same assembly to simplify the use of the SW2URDF plugin, so modelling them as parts straight in the assembly seemed reasonable.

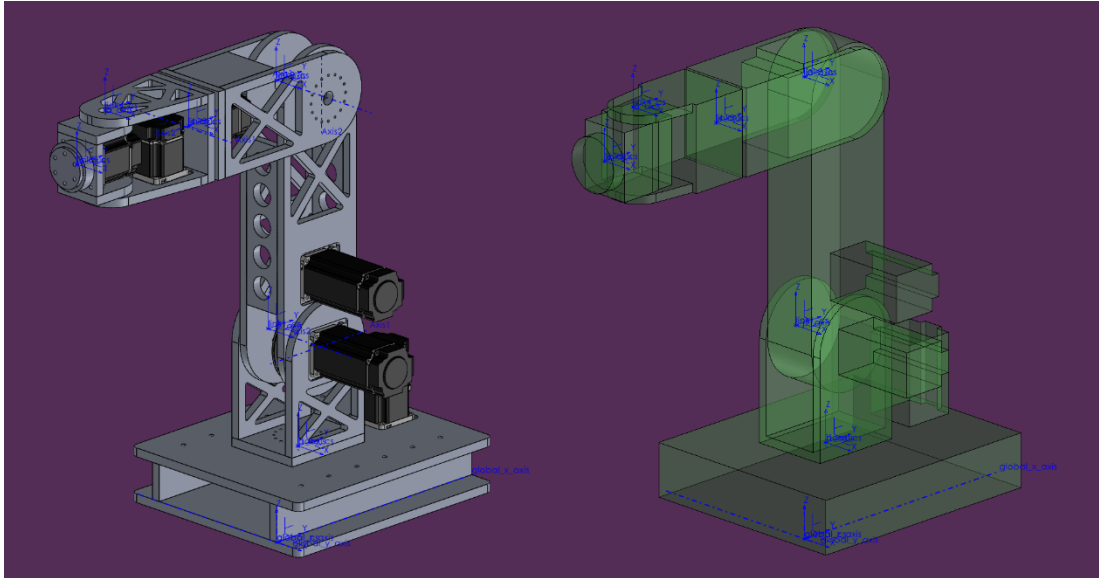


Figure 9. Different mesh files ready for export, visual on the left and collision on the right. Joint axis and link coordinate system placements can be seen in blue.

5.6 Exporting to URDF

After setting up the model in SolidWorks, the plugin can be launched in SolidWorks menu under 'Tools – Export as URDF'. On some older versions of SolidWorks, the same export option is found in the 'File' menu. The process of connecting the links and joints goes from parent to child, so in this case the first link is the base. Link name, joint name to parent link, coordinate system of the link, reference axis of the joint, joint type and the parts that belong to that link must be selected for each link. In the link configuration panel, the unlabelled field at the bottom is the number of child links, increasing it from zero unlocks the next child in the link-joint chain to be selected.

After going through the whole kinematic chain of the robot with the procedure described above, configuration can be continued with the 'Preview and export' button, which launches a new window. In the 'joint properties' configurator, there are multiple customisation options per joint. It is possible to set limits for joint movement, effort(torque) and velocity, add friction and damping, and more. In this case, only joint movement and effort limits were set, all other values were left at default values.

After all the joints are configured, pressing 'Next – Export URDF and Meshes' opens a 'save as' prompt, where the ROS package name can be entered. It is also worth noting for setups with separate visual and collision files: This link and joint configuration process is done twice, once with the visual files and saved for example in 'robot_combined_mesh' package and another time for the collision files and saved for example in 'robot_collision_mesh' package. These are exported separately, because selecting parts for a link in the configuration will result in one mesh file per link. To get two separate files per link, the configuration is run twice. Fortunately, if using the same assembly for all files, the SW2URDF plugin remembers the link and joint configurations from the first export and only the parts to generate mesh files need to be changed.

As stated before, the SolidWorks plugin is dated and it produces ROS1 packages, that will not work in ROS2 as is. The two packages were first combined for convenience, so that the mesh files from the collision package were moved over to the combined package, and then in the URDF file, paths in the collision tags were linked to the appropriate collision mesh files. After that the single package was uploaded GitHub for easy access and safe storage for the ROS2 build phase.

6 IMPORTING THE ROBOT INTO ROS2

Before starting the robot configuration, ROS2 environment must be setup. First option for the environment was ROS2 Humble in Ubuntu 22.04, simply because of system stability due to the longer development time since May 2022. Unfortunately, I ran into display driver compatibility problems when trying to set up gazebo simulation at a later stage of the process. The compatibility problems were persistent enough to warrant a change to ROS2 Jazzy in Ubuntu 24.04, which is the most recent version. ROS2 installation documentation is quite extensive, and the process went as described in the guide for Ubuntu installation. (Open Robotics, n.d.) Next step is to setup MoveIt2 for visualising

and future motion control for the robot. Once again, the MoveIt2 installation guide was thorough and worked as described. There is a time-consuming build operation at the end, which is also mentioned in the guide. (PickNik Robotics, n.d.a) If the build process fails due to hardware limitations, such as running out of memory, this can be alleviated by running the build process either sequentially or limiting parallel jobs (Figure 10).

```
~/ws_moveit$ colcon build --executor sequential #install in sequence
~/ws_moveit$ colcon build --parallel-workers 4 #max 4 parallel processes
```

Figure 10. Colcon build commands for sequential building and limiting parallel processes.

It is worth mentioning, that a lot has changed from ROS1 to ROS2, both in the backend, but also in terminal use of the system. Sourcing, build system and much more have been changed. This build system change is one of the reasons why we cannot use the ROS1 package, that was created by the SW2URDF plugin, as is. Instead, a 'm1u_description' package is created to hold the main URDF and mesh files to describe our robot in later steps. The example robot was named 'm1u'. The 'm1u_description' package is created with the necessary dependencies for later use, already linked for the build system. The package is created into the same ROS2 workspace folder that was created during the MoveIt2 setup. After creating the package, the previous GitHub repo containing the URDF and mesh files is cloned to the system, and the URDF and mesh files are copied to the 'm1u_description' respective URDF and MESHES folders. Because we are in the previously created workspace with all the MoveIt2 packages, we can use the 'select packages' option in the build command to only build our new 'miu_description' package (Figure 11).

```
~/ws_moveit$ colcon build --packages-select m1u_description
```

Figure 11. Using '--packages-select' option lets us only build selected packages.

After building the package, the workspace must be sourced and after, the MoveIt2 setup assistant can be started (Figure 12).

```
~/ws_moveit$ source install/setup.bash
~/ws_moveit$ ros2 launch moveit_setup_assistant setup_assistant.launch.py
```

Figure 12. Sourcing the workspace and launching MoveIt2 setup assistant to start configuring the robot.

6.1 MoveIt2 setup assistant

MoveIt2 setup assistant (MSA) is a tool to create MoveIt2 packages from robot description files. MoveIt2 setup assistant documentation goes over the process of creating a ROS2 MoveIt2 package in detail and it worked as expected. The process goes through generating a self-collision matrix, configuring movement planning groups and different controllers and poses (Figure 13). In the end, the setup assistant generates a new ROS2 package, that includes the MoveIt2 configuration files and launch files, that references the previous 'm1u_description' package for the URDF and mesh files. The example of the setup assistant documentation uses a articulated robot called Panda, which makes following along with another articulated robot very straight forward. (PickNik Robotics, n.d.b).

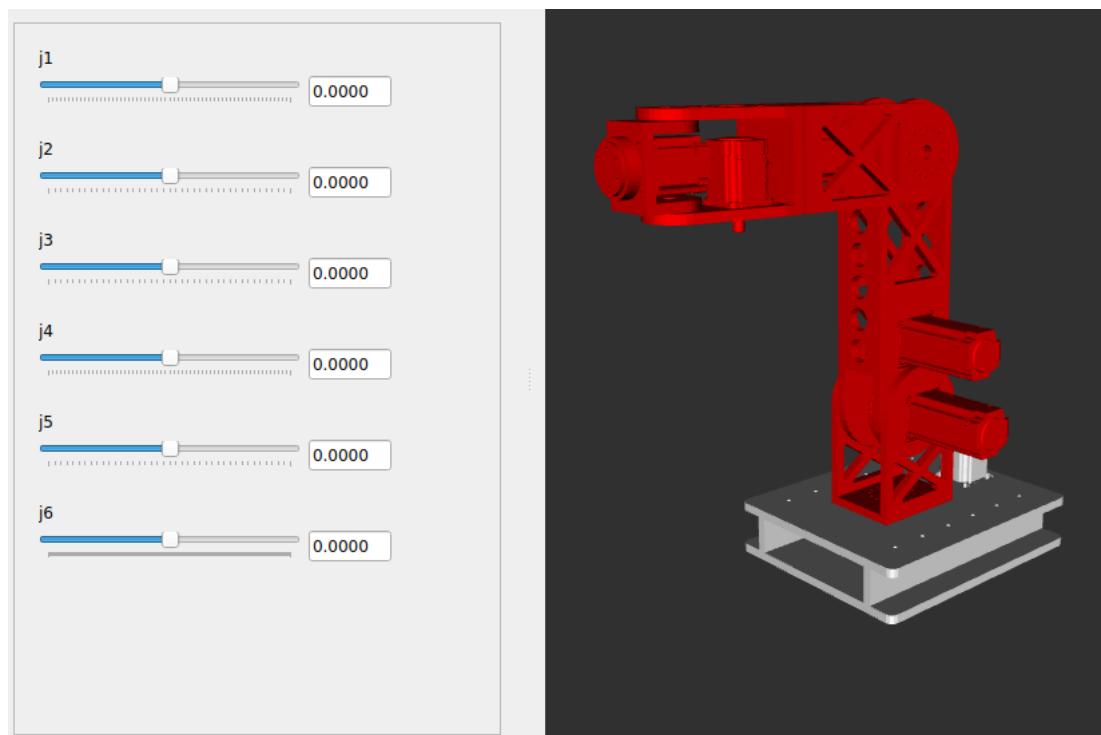


Figure 13. Pose configuration in MoveIt2 setup assistant.

6.1.1 MoveIt2 demo in RViz2

After the 'm1u_moveit' package has been created with the setup assistant, it must be built, sourced and then a visualisation demo can be launched (Figure 14).

```
~/ws_moveit$ colcon build --packages-select m1u_moveit  
~/ws_moveit$ source install/setup.bash  
~/ws_moveit$ ros2 launch m1u_moveit demo.launch.py
```

Figure 14. Building the package, sourcing the workspace and launching the visualisation demonstration in RViz2.

The 'demo.launch.py' allows to try out movement planning in RViz2 without actual simulation or physical robot required. It contains the whole interface one would have with a simulated or a physical robot, without the need of configuring the controllers and sensors at this stage. The demonstration creates and uses 'fake hardware' to be used between ROS2 nodes and topics for the movement planning and execution.

6.2 Simulating the robot in Gazebo

Gazebo is one of the simulators available for ROS2. In this case, with ROS2 Jazzy, the correct version to use is Gazebo Harmonic. Gazebo is developed by Open Robotics, same as ROS2, so it integrates well into our existing system. Main reason for choosing Gazebo as the simulation tool was simply my previous experience with older versions of Gazebo in ROS1 environment. Gazebo offers simulation for both, physics to the whole scene and robot, like gravity and collisions, and also the possibility to simulate various types of sensors, like inertial measurement units (IMUs), lidars and cameras. Our simulation is only a physics simulation of the robot; no sensors were added.

For the simulation, another ROS2 package was created. The idea behind splitting and containerization of different packages of the robot into description, MoveIt2 and now simulation, is modularity for future use. This way, there is no need to, for example, add Gazebo simulation related configurations into the

description package of the robot. With a physical robot in the future, the simulation package is not needed, and the simulation related configurations made in the description package would need to be replaced with ones corresponding to the real-world robot.

6.2.1 Setting up Gazebo simulation

Creating the Gazebo simulation configuration started with creating a new ROS2 package that imports from the original description package and uses configurations from the MoveIt2 package. The main things to setup the robot for Gazebo, were to create a wrapper xacro file and controller configuration file. The simulation package xacro imports the original URDF file from the description package, which contains the information of the robot links and joints. On top of the import, per joint command and state interfaces are added to the xacro file along with a Gazebo controller plugin. A separate controller configuration file is also created, where the controller types and some parameters like update rate and use of simulated time are defined. Finally, a launch file is created, which launches Gazebo, spawns the simulated robot, its controllers and state publishers, and launches a RViz2 instance with the connected controllers, from the separate MoveIt2 package.

6.2.2 Troubleshooting the Gazebo environment

While the configuration process to achieve a working Gazebo simulation with MoveIt2 motion planning seems quite straightforward in theory, I faced by far the most challenges in this stage, compared to the rest of the process. As I mentioned before in the beginning of Chapter 6, the first environment in Ubuntu 22.04 was abandoned due to graphics driver issues. On the final environment in Ubuntu 24.04 and ROS2 Jazzy, the Gazebo demonstrations that are included with the initial installation worked right out of the box, without graphics driver issues. A challenge that remains in the setup, is the fact that Gazebo, for some reason, does not include the built ROS2 workspace folders in the resource paths that it uses, and thus it does not find the mesh files for the robot

links. This is currently bypassed by manually appending the workspace folders into the path, and it needs to be done each time the system is restarted. The terminal commands for this are included in the setup and use guide in the GitHub repository of this project.

Another problem that took quite a lot of time to solve, was syncing the time between the Gazebo simulation and the MoveIt2 motion planning. The offset between the times of the two parts of the system resulted in tf2 transform errors between the links of the robot, resulting in failed motion executions and simulation crashes. The problem was ultimately that the timestamp in the MoveIt2 system was not updating correctly. Once explicitly forcing both Gazebo and MoveIt2 to use the same simulated time, the problem was resolved.

6.2.3 Gazebo and MoveIt2 interaction

Simplified and at a high level, the control system of the MoveIt2 and Gazebo simulated robot works as follows. The Simulation package defines the controllers used in the Gazebo simulation, in this case there is one controller, called 'JointTrajectoryController', to take in goal positions for the different joints of the robot, and another controller, called 'JointStateBroadcaster', that reads the current joint states from the Gazebo simulation and publishes them. MoveIt2 motion planning generates a series of joint positions, which are then used by the JointTrajectoryController to move the robot. At the same time, the JointStateBroadcaster is publishing the current state of the simulated robot's joints. These joint states are used by the 'robot_state_publisher' ROS2 node to determine the pose of the robot. RViz2 uses this robot pose information to visualise the current state of the robot. A simple diagram of the flow is described in Figure 15. A detailed version with all the actual ROS2 nodes and topics can be viewed by launching a ROS computation graph (rqt_graph) while the robot simulation is running. To reiterate the role of RViz2 in this control system: RViz2 is here to mainly provide an easy-to-use graphical interface to use MoveIt2 motion planning with, and further, listen to the state of the robot

published by the robot_state_publisher node, and visualise the current position of the robot (Figure 16).

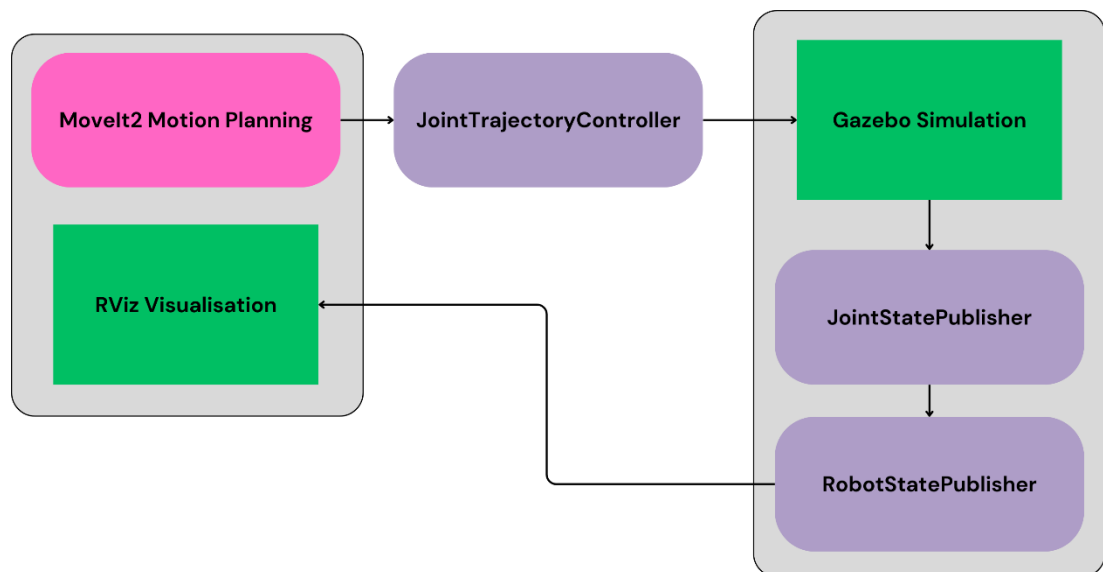


Figure 15. A simplified flowchart of the motion planning of the simulated robot.

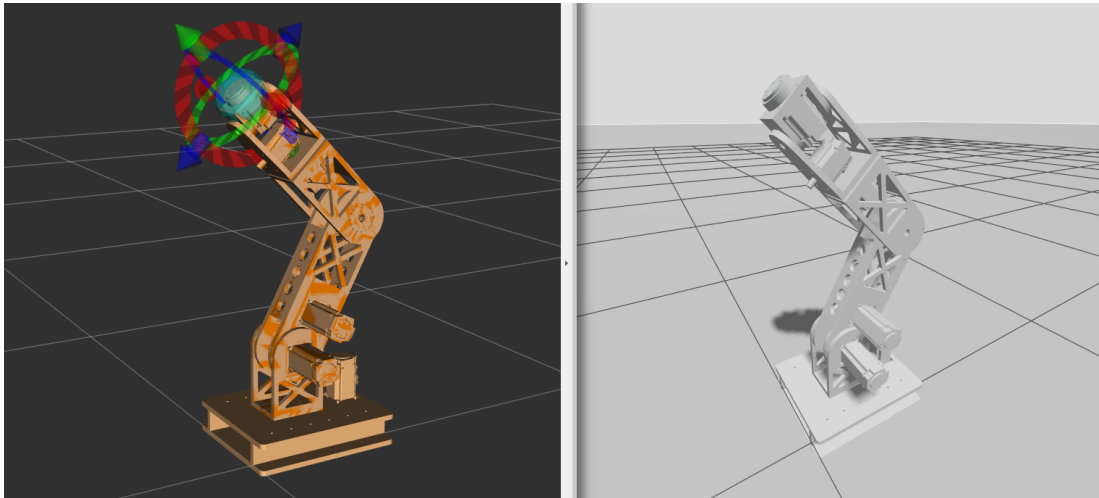


Figure 16. A view of the robot after planning and executing a movement, Movelt2 visualisation on the left and the simulated robot in Gazebo on the right.

7 RESULTS AND CONCLUSIONS

7.1 Evaluation of design choices

I feel like the design of the robot in the current state has its strengths and weaknesses. On one hand, it is quite simple to manufacture the parts of the robot, as the frame was designed to be CNC cut and machined from a plate stock, with very reasonable tools in modern manufacturing environments, that can for example be found in our mechanical engineering laboratory at Satakunta University of Applied Sciences. On top of the easy to manufacture parts, sticking to components with good availability caters well to supply chain resilience, which in my opinion is an important consideration in the current state of the world. The open-source ideology and community-based projects result in good availability for the systems, which in general is very positive from the supply chain resilience perspective, and in my opinion should be considered more in designs.

On the other hand, from a mechanical standpoint, I am not totally pleased with the current state of the robot design. I would like to do at least a partial redesign of the upper part of the robot arm, reconsidering the motor placements, need for gearboxes and possible power transmission solutions for the joints 4, 5 and 6. My main worry with the upper part of the robot is the cable management and routing for the joints 5 and 6. Redesigning the joint 4 to be a hollow shaft joint where the cables could be routed through the middle of the rotational joint would make cabling for joints 5 and 6 simpler. Also, while it allows a convenient motor placement, I would like to get rid of the belt drive in joint 1. For actual use I would also design lightweight 3D printed panels for the frame, to block access to the drive systems of the robot, and to make the robot visually cleaner.

While the design did not end up totally production ready during the thesis work, I still think the main choices in the design were very reasonable. With rigid structure and accurate motors and gearboxes, when compared to other open-

source projects, I feel like the design bridges the wide gap between current open-source projects and industrial robots quite well. The model development will continue, until a production ready state is reached.

7.2 Objectives, results and outcomes

The main objective of the thesis was to create the 3D model of the robot and implement a ROS2 Gazebo simulation with MoveIt2 motion planning. This objective was reached, and a simulation was created successfully. The resources needed to run the simulation, namely the three different ROS2 packages are available in the GitHub repository of this project, along with instructions for setup and use. In my opinion, this part of the project was a success, and the system can be used to demonstrate and teach the ROS2 setup of a custom robot in a simulation and integrating it with the MoveIt2 motion planning package. The GitHub repository of the project can be found in Appendix 2.

The secondary objective was to create technical drawings and a bill of materials for future manufacturing. In hindsight this goal may not have been fully relevant, as the intended manufacturing methods of the parts are CNC cutting and milling, the technical drawings can be seen as redundant, as the 3D models would be used to generate the toolpaths for manufacturing. Nevertheless, this goal was not fully reached, and although the technical drawings can be exported from the model in the current state, they will not be included in the thesis work, aside from one general drawing showing the dimensions of the robot in Appendix 1. This is due to my personal will to refine the design before calling it ready for manufacture. 3D files of the current model are included in the GitHub repository of the project, and the technical drawings can be exported from those models. The model files in the repository will be updated as the model gets refined and finally reaches the state where I personally see it ready for production.

7.3 Future development and application

As mentioned in the previous chapter, the development of the robot will continue until a state is reached where I feel confident that it will be reasonable to manufacture and use in a teaching environment. This will involve design of the control system in detail, both hardware, including cabling and a controller enclosure, and software. On the simulation side of the project, next steps could be incorporating an end effector into the simulated robot and setting up a task for it, for example a simple pick and place to start with.

7.4 Personal reflections and learning outcomes

Looking back at the whole thesis project, I feel like I learned a lot during it, and in various aspects. While the mechanical design of the system is simple, it became from design for manufacturing (DFM) reasons. The benefits of simplicity and DFM principles are often forgotten in my own design process, when trying to create the 'perfect' part that has extra features and often ends up more complicated than necessary and difficult to produce. Regarding the latter half of the thesis: the ROS2 setup, Gazebo simulation and MoveIt2 motion planning, it was a very educational process. I had quite little previous experience with ROS2, and my background was mainly in ROS1. The scale of difference between these two systems took me by surprise at first, but I am happy that I stepped into the new platform, and did not use the end-of-life ROS1, which would have been the more familiar and easier platform. Looking ahead, ROS2 is something that I wish to understand better and be able to use more in the future, and this project was a great introduction to many of the aspects of ROS2 as a development environment.

REFERENCES

- ABB Robotics. (2019). [Schematic of the workspace of an ABB IRB 120 robot]. [Image]. https://library.e.abb.com/public/7aa0711a20fa41c49a8fdf3bbc3d5bb0/IRB120-Rev.J-ROBO149EN_D.pdf
- ABB Robotics. (2020, May 14). [ABB IRB 390 FlexPacker robot]. [Image]. <https://new.abb.com/news/detail/62221/robotic-picking-and-packing-portfolio>
- ABB Robotics. (2024, December 18). Autonomous Mobile Robots (AMRs) — ABB AMR/AGV robots. Retrieved 19 May 2025, from <https://new.abb.com/products/robotics/autonomous-mobile-robots>
- Allinson, M. (2024, September 5). Bearings for Robot Arms: The Complete Guide. Robotics & Automation News. <https://roboticsandautomationnews.com/2024/09/05/bearings-for-robot-arms-the-complete-guide/85436/>
- Arrow Electronics. (2022, June 10). [Stepper motor working principle]. [Image]. <https://www.arrow.com/en/research-and-events/articles/stepper-motor-basics-explained>
- Bernier, C. (2021, October 28). Cartesian Robots: Guide to The Most Scalable Robot Technology. HowToRobot. <http://howtorobot.com/expert-insight/cartesian-robots>
- Boschetti, G., & Sinico, T. (2023). Performance Comparison of Two Architectures of 6R Articulated Robots. *Machines*, 11(2), 306. <https://doi.org/10.3390/machines11020306>
- Choi, H., Crump, C., Duriez, C., Elmquist, A., Hager, G., Han, D., Hearl, F., Hodgins, J., Jain, A., Leve, F., Li, C., Meier, F., Negrut, D., Righetti, L., Rodriguez, A., Tan, J., & Trinkle, J. (2021). On the use of simulation in robotics: Opportunities, challenges, and suggestions for moving forward. *Proceedings of the National Academy of Sciences*. 118(6). <https://doi.org/10.1073/pnas.1907856118>
- Cohen, R., Lipton, M. G., Dai, M. Q., & Benhabib, B. (1992). Conceptual Design of a Modular Robot. *Journal of Mechanical Design*. 114(2), 117-125. <https://doi.org/10.1115/1.2916904>
- Craig, J. J. (2005). *Introduction to Robotics: Mechanics and Control* (third edition). Pearson Prentice Hall.

Devasia, A. (2023). An Introduction to Types of Closed-loop Motor Control. Control.Com. Retrieved 29 April 2025, from <https://control.com/technical-articles/in-introduction-to-types-of-closed-loop-motor-control/>

Hale, L. (1999). Principles and techniques for designing precision machines. [Ph.D. Thesis, University of California]. <https://doi.org/10.2172/8431>

Hanbury, P., Dutt, A., & Veratti, E. (2025). Humanoid Robots at Work: What Executives Need to Know. Bain. Retrieved 19 May 2025, from <https://www.bain.com/insights/humanoid-robots-at-work-what-executives-need-to-know/>

International Federation of Robotics. (2024, September 24). Record of 4 Million Robots in Factories Worldwide. <https://ifr.org/ifr-press-releases/news/record-of-4-million-robots-working-in-factories-worldwide>

International Federation of Robotics. (2025, January 22). TOP 5 Global Robotics Trends 2025. <https://ifr.org/ifr-press-releases/news/top-5-global-robotics-trends-2025>

ISO 12100:2010(en). (2010). Safety of machinery - General principles for design – Risk assessment and risk reduction. International Organization for Standardisation. <https://www.iso.org/obp/ui/#iso:std:iso:12100>

Kingstar. (n.d.). [SCARA robot joint configuration]. [Image]. https://help.kingstar.com/product_help/KS/Content/Resources/Images/Concept/Robots/SCARA/SCARA_Robot.png

Malm, T. (2022). Safety design process for collaborative robots (VTT-R-00062-22). VTT Technical Research Centre of Finland. https://cris.vtt.fi/ws/portalfiles/portal/58293171/VTT_Sqn_CobotSafety2022.pdf

Mecademic Industrial Robotics. (n.d.) [Description of different types of singularities in a 6DOF arm robot]. [Image]. <https://mecademic.com/insights/academic-tutorials/what-are-singularities-6-axis-robot-arm/>

Nimtz, A., & Fichtner, M. (2024, December 19). A look at the Machinery Regulation (EU) 2023/1230. iSAX. <https://www.isax.com/en/about/blog/machinery-regulation-2023-1230/>

Niryo. (2024, July 17). 6 criteria to choose the right motor for a jointed arm robot. <https://niryo.com/6-criteria-choose-right-motor-jointed-arm-robot/>

Open Robotics. (n.d.). Ubuntu (deb packages)—ROS 2 Documentation: Jazzy documentation. Retrieved 15 May 2025, from <https://docs.ros.org/en/jazzy/Installation/Ubuntu-Install-Debs.html>

Owen-Hill, A. (2022a, November 30). Robot Singularities: What Are They and How to Beat Them. RoboDK Blog. Retrieved 28 April 2025, from <https://robodk.com/blog/robot-singularities/>

Owen-Hill, A. (2022b, November 8). 7 Types of Robots You Should Know About. RoboDK Blog. Retrieved 19 May 2025, from <https://robodk.com/blog/7-types-of-robots-you-should-know-about/>

PickNik Robotics. (n.d.a). Getting Started—MoveIt Documentation: Rolling documentation. Retrieved 15 May 2025, from https://moveit.picknik.ai/main/doc/tutorials/getting_started/getting_started.html

PickNik Robotics. (n.d.b). MoveIt Setup Assistant—MoveIt Documentation: Rolling documentation. Retrieved 15 May 2025, from https://moveit.picknik.ai/main/doc/examples/setup_assistant/setup_assistant_tutorial.html#step-1-start

Regulation 2023/1230/EU of the European Parliament and of the Council of 14 June 2023 on machinery and repealing Directive 2006/42/EC of the European Parliament and of the Council and Council Directive 73/361/EEC. <http://data.europa.eu/eli/reg/2023/1230/oj>

Rocco, D. (2018, April 1). Choosing the Right Brake for Robotic Applications. <https://www.techbriefs.com/component/content/article/28812-choosing-the-right-brake-for-robotic-applications>

Siciliano, B., & Khatib, O. (2016). Springer Handbook of Robotics (second edition). Springer Cham. <https://doi.org/10.1007/978-3-319-32552-1>

Sntoom. (2024, September 15). Comprehensive Guide to Different Types of Motors Used in Robotics. <https://sntoom.net/comprehensive-guide-to-different-types-of-motors-used-in-robotics/>

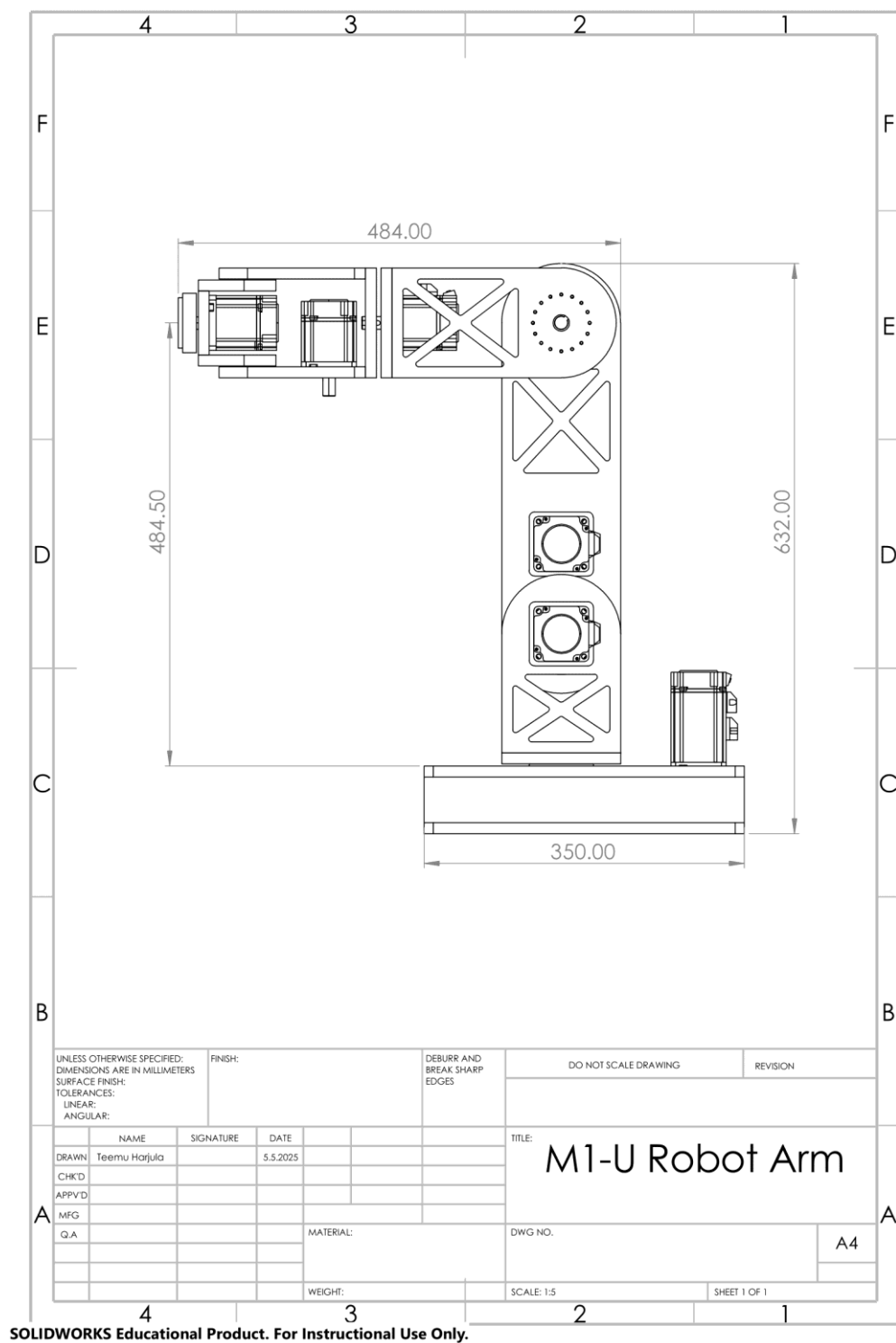
Sunrise Metal. (2024, November 19). What is a good material for a robot? <https://www.sunrise-metal.com/what-is-good-material-for-robot>

Tec-science. (2018, October 8). Overview of gear types. [Image]. <https://www.tec-science.com/mechanical-power-transmission/gear-types/cylindrical-gears/>

Walter, M. (2016, August 29). How Accurate Is Microstepping Really? Hackaday. <https://hackaday.com/2016/08/29/how-accurate-is-microstepping-really/>

Wang, Y. (2022, October 6). Are Cobots Safe to Work Side by Side with Humans? IDTechEx. <https://www.idtechex.com/en/research-article/are-cobots-safe-to-work-side-by-side-with-humans/27885>

APPENDIX 1: OVERVIEW OF THE ROBOT



APPENDIX 2: GITHUB REPOSITORY OF THE PROJECT

https://github.com/tharj/m1u_configuration